

Paytm SSE Round 1 - Java & Spring Boot Deep Dive

1. HashMap Internals

How does `put()` work internally?

When you call `map.put("name", "Satjit")`, here's exactly what happens under the hood:

Step 1 — Hash the key

```
int hash = key.hashCode(); // e.g., "name".hashCode() → 3373707
hash = hash ^ (hash >> 16); // Spread high bits into low bits (reduces collisions)
```

The XOR with the upper 16 bits is called **hash perturbation**. It ensures that keys whose hashCodes differ only in higher bits don't all land in the same bucket.

Step 2 — Find the bucket

```
int bucketIndex = hash & (capacity - 1); // Same as hash % capacity, but faster
```

The default capacity is 16, so there are 16 buckets (indices 0-15).

Step 3 — Place the entry

- If the bucket is empty → create a new `Node` and place it there.
- If the bucket already has entries → **collision handling** (see below).
- If the key already exists (checked via `.equals()`) → overwrite the value.

Step 4 — Check load and resize

```
If (size > capacity * loadFactor) → resize to 2x capacity
Default loadFactor = 0.75 So: 16 * 0.75 = 12 →
resize at 13th entry
```

What happens during a collision?

Before Java 8, collisions were handled by a **linked list** — new entries were just added to the chain at that bucket index.

```
Bucket 5: [Entry A] → [Entry B] → [Entry C] (Linked List)
```

The problem: If many keys hash to the same bucket, the linked list grows long, and `get()` degrades from $O(1)$ to $O(n)$.

What is the Treeify Threshold? (Java 8+)

Java 8 introduced a brilliant optimization:

*When a single bucket's linked list grows to **8 or more entries** (and the total table capacity is at least 64), the linked list is converted into a **Red-Black Tree**.*

Bucket 5: [Entry A] → [Entry B] → ... → [Entry H] ↓ TREEIFY (8+ nodes) Bucket 5: [Entry D] / \ [Entry B]
[Entry F] / \ / \ [A] [C] [E] [G,H]

This improves worst-case lookup from $O(n)$ to $O(\log n)$.

Untreeify Threshold: When the tree shrinks to **6 or fewer** nodes (due to removals), it converts back to a linked list. The gap between 8 and 6 prevents constant flipping (hysteresis).

The Full Picture (Memory Layout)

```
HashMap (capacity=16, size=5, loadFactor=0.75) ████████████████████████████████████████  
Bucket 0: null █ █ Bucket 1: [K=city, V=Delhi] █ █ Bucket 2: null █ █ Bucket 3: [K=name, V=Satjit] →  
[K=age, V=26] ← collision (linked list) █ █ Bucket 4: null █ █ Bucket 5: [K=role, V=SDE] █ █ ... █ █ Bucket  
12: [K=company, V=Amdocs] █ █ ... █ █ Bucket 15: null █ █  
████████████████████████████████████████
```

Interview One-Liner

"HashMap uses an array of buckets. Each key is hashed, then placed in the bucket at $\text{hash} \& (n-1)$. Collisions form a linked list, which treeifies into a Red-Black Tree at 8 nodes in Java 8+ for $O(\log n)$ worst-case lookup."

2. Concurrency — Fail-Fast vs Fail-Safe Iterators

Fail-Fast Iterators (ArrayList, HashMap)

```
List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C")); Iterator<String> it = list.iterator();
list.add("D"); // Structural modification WHILE iterating it.next(); // BOOM →
ConcurrentModificationException
```

How it works: The collection maintains a `modCount` (modification counter). When you create an iterator, it snapshots the current `modCount`. On every `next()` call, it checks if `modCount` has changed. If yes → exception.

Real World: Think of a shared Google Doc. You're reading paragraph 3, and someone deletes paragraph 2. Your reading position is now invalid — the system throws an error rather than give you garbage data.

Fail-Safe Iterators (ConcurrentHashMap, CopyOnWriteArrayList)

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>(); map.put("A", 1); map.put("B", 2); for
(String key : map.keySet()) { map.put("C", 3); // No exception! Works fine. }
```

How it works: These iterators work on a **snapshot or a weakly-consistent view** of the data. Modifications don't affect the ongoing iteration.

Trade-off: You might not see the latest modifications during iteration.

Quick Comparison

Feature	Fail-Fast	Fail-Safe
Exception on modification?	Yes (ConcurrentModificationException)	No
Works on	Original collection	Clone/snapshot
Examples	ArrayList, HashMap, HashSet	ConcurrentHashMap, CopyOnWriteArrayList
Memory overhead	None	Higher (clone/snapshot)
Use case	Single-threaded iteration	Multi-threaded environments

3. ConcurrentHashMap — How It Achieves Thread Safety

The Evolution

Java 7: Segment Locking

- The map was divided into 16 **segments**, each with its own lock.
- Writing to Segment 3 doesn't block reading/writing Segment 7.
- 16 threads could write simultaneously (one per segment).

Java 8+: CAS + Synchronized per Bucket (Current Approach)

- No more segments. Locking is at the **individual bucket level**.
- Uses **CAS (Compare-And-Swap)** for inserting into an empty bucket (lock-free).
- Uses **synchronized on the first node** of a bucket for inserting into an occupied bucket.

Bucket 0: empty → CAS to insert (no locking at all) Bucket 3: [A] → [B] → synchronized on node [A] to add [C] Bucket 7: empty → CAS to insert

Why Not Just Use Collections.synchronizedMap()?

```
Map<String, Integer> syncMap = Collections.synchronizedMap(new HashMap<>());
```

This wraps every single method in a `synchronized` block on the **same mutex**. So reading blocks writing, writing blocks reading, and only one thread can access the map at a time. It's a sledgehammer approach.

ConcurrentHashMap is a scalpel:

- Reads are completely lock-free (using `volatile` reads)
- Writes only lock the specific bucket being modified
- Multiple threads can read and write simultaneously (to different buckets)

Real World Analogy

- `synchronizedMap` = One cashier at a bank. Everyone waits in one line.

- ## 4. JVM — The G1 Garbage Collector

Step 1: Observe the symptoms

- `OutOfMemoryError: Java heap space`
- Heap usage grows continuously and never drops after GC
- App gets progressively slower

Step 2: Enable GC logging

```
java -Xlog:gc*:file=gc.log:time -jar app.jar
```

Look for Full GC events that don't reclaim much memory.

Step 3: Take a heap dump

```
# On a running app jmap -dump:live,format=b,file=heap.hprof <PID> # Or configure auto-dump on OOM java
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/ -jar app.jar
```

Step 4: Analyze with tools

- **Eclipse MAT (Memory Analyzer Tool)**: Open the heap dump, check the "Leak Suspects" report.
- **VisualVM**: Real-time monitoring of heap, classes, threads.

Step 5: Common culprits in Spring Boot

- **Static collections** that keep growing (e.g., a static Map used as cache without eviction)
- **Unclosed resources** — DB connections, HTTP clients, streams
- **Session-scoped beans** holding large objects
- **Listeners/observers** not being deregistered
- **ThreadLocal** variables not cleaned up (common in web apps with thread pools)

5. Spring Boot — @Component vs @Service vs @Repository

The Short Answer

Functionally, they're **almost identical** — all three register the class as a Spring-managed bean. The difference is **semantic intent** and a few special behaviors.

The Hierarchy

```
@Component ← Generic bean ■■■ @Service ← Business logic bean ■■■ @Repository ← Data access bean (has extra
magic) ■■■ @Controller ← Web layer bean
```

`@Service` and `@Repository` are **specialized versions of @Component**. If you look at their source code:

```
@Component // ← @Service IS a @Component public @interface Service { ... } @Component // ← @Repository IS a
@Component public @interface Repository { ... }
```

Why Use Different Annotations?

1. Readability & Intent

```
@Service public class PaymentService { ... } // Clearly business logic
@Repository public class PaymentRepository { ... } // Clearly data access
```

2. @Repository has a special superpower: Exception Translation

Spring automatically wraps any persistence-specific exceptions (like `SQLException`, Hibernate's `PersistenceException`) thrown from `@Repository` classes into Spring's unified `DataAccessException` hierarchy.

```
@Repository public class UserRepository {
    public User findById(Long id) {
        // If this throws a JDBC
        // SQLIntegrityConstraintViolationException,
        // Spring automatically translates it to
        // → DataIntegrityViolationException (a Spring DataAccessException)
    }
}
```

This means your service layer doesn't need to catch vendor-specific exceptions.

3. AOP Targeting

You can write an aspect that targets only `@Service` beans:

```
@Around("@within(org.springframework.stereotype.Service)")
public Object logServiceMethods(ProceedingJoinPoint pjp) { ... }
```

6. How @Transactional Works Internally (AOP Proxies)

The Simple Explanation

When you annotate a method with `@Transactional`, Spring doesn't modify your class. Instead, it creates a **proxy** that wraps your class. All calls go through this proxy, which manages the transaction.

```
Controller → [Transaction Proxy] → YourService.transfer()
    ■■■■ BEGIN TRANSACTION ■■■■ Call actual transfer() method
    ■■■■ If success → COMMIT ■■■■ If exception → ROLLBACK
```

How the Proxy is Created

Spring uses two approaches:

1. **JDK Dynamic Proxy** — If your class implements an interface
2. **CGLIB Proxy** — If your class has no interface (creates a subclass at runtime)

```
@Service public class PaymentService {
    @Transactional public void transfer(Account from, Account to,
    BigDecimal amount) {
        from.debit(amount);
        to.credit(amount);
        // If any exception occurs here, the WHOLE thing rolls back
    }
}
```

What Spring actually creates at runtime (conceptually):

```
public class PaymentService$$EnhancerByCGLIB extends PaymentService {
    @Override public void transfer(Account from, Account to,
    BigDecimal amount) {
        TransactionStatus tx = transactionManager.getTransaction(...);
        try {
            super.transfer(from, to, amount);
            // Call the REAL method
            transactionManager.commit(tx);
        } catch (RuntimeException e) {
            transactionManager.rollback(tx);
            throw e;
        }
    }
}
```

The #1 Gotcha: Self-Invocation

```
@Service public class OrderService { public void processOrder(Order order) { validateOrder(order);
saveOrder(order); // THIS CALL BYPASSES THE PROXY! } @Transactional public void saveOrder(Order order) {
orderRepo.save(order); } }
```

When `processOrder()` calls `saveOrder()` internally, it's a direct method call on this — it **doesn't go through the proxy**. So `@Transactional` has NO effect.

Fix: Inject self or use `TransactionTemplate`:

```
@Service public class OrderService { @Autowired private OrderService self; // Inject the proxy public void
processOrder(Order order) { validateOrder(order); self.saveOrder(order); // Goes through the proxy →
@Transactional works } }
```

Key @Transactional Attributes

Attribute	Default	Meaning
<code>propagation</code>	REQUIRED	Join existing tx, or create new one
<code>isolation</code>	DEFAULT (DB default)	Read isolation level
<code>rollbackFor</code>	RuntimeException only	Which exceptions trigger rollback
<code>readOnly</code>	false	Optimization hint for the DB
<code>timeout</code>	-1 (none)	Max seconds for the transaction

Real World Example

At Paytm, a money transfer needs to be atomic:

```
@Transactional( rollbackFor = Exception.class, // Rollback on ANY exception, not just Runtime isolation =
Isolation.SERIALIZABLE, // Prevent double-spending timeout = 5 // Fail fast if DB is slow ) public void
transfer(String fromWallet, String toWallet, BigDecimal amount) { walletRepo.debit(fromWallet, amount);
walletRepo.credit(toWallet, amount); transactionLogRepo.log(fromWallet, toWallet, amount); }
```

7. How to Handle Circular Dependencies in Spring

What is a Circular Dependency?

```
@Service public class ServiceA { @Autowired private ServiceB serviceB; // A needs B } @Service public class
ServiceB { @Autowired private ServiceA serviceA; // B needs A }
```

Spring can't create A without B, and can't create B without A. Deadlock.

How Spring Used to Handle It (Before 6.0)

Spring had a **three-level cache** for singletons:

1. `singletonObjects` — Fully initialized beans
2. `earlySingletonObjects` — Partially initialized beans (constructed but not fully wired)
3. `singletonFactories` — Factories that can produce early references

When creating A, Spring would:

1. Instantiate A (call constructor)
2. Put an early reference to A in the cache
3. Start wiring A's dependencies → needs B
4. Create B, wire B's dependencies → needs A → finds early reference in cache
5. B is fully created
6. Complete wiring A

This only works with field/setter injection, NOT constructor injection.

Spring 6.0+ (Stricter by Default)

Spring 6.0 disabled circular dependency support by default. You get:

```
BeanCurrentlyInCreationException: Error creating bean 'serviceA': Requested bean is currently in creation - is there an unresolvable circular reference?
```

Solutions (Best to Worst)

1. **Redesign (Best)** — Circular dependencies usually indicate a design problem. Extract the shared logic into a third service.

```
@Service public class ServiceA { @Autowired private SharedService shared; } @Service public class ServiceB { @Autowired private SharedService shared; } @Service public class SharedService { // Common logic that both A and B needed from each other }
```

2. **Use @Lazy on one side**

```
@Service public class ServiceA { @Autowired @Lazy private ServiceB serviceB; // Spring injects a PROXY, not the real bean }
```

The proxy is created immediately (breaking the cycle), but the real ServiceB is only resolved when a method on it is actually called.

3. **Use an event-based approach** — Instead of A calling B directly, A publishes an event that B listens to.

```
@Service public class ServiceA { @Autowired private ApplicationEventPublisher publisher; public void doSomething() { publisher.publishEvent(new OrderCreatedEvent(this, orderId)); } } @Service public class ServiceB { @EventListener public void handleOrderCreated(OrderCreatedEvent event) { // React to A's action without A depending on B } }
```

Quick Revision Table

Topic	Key Point to Remember
HashMap put()	hash → bucket index → check collision → treeify at 8
Treeify Threshold	8 nodes → Red-Black Tree, 6 → back to LinkedList
Fail-Fast	ConcurrentModificationException, checks modCount
Fail-Safe	Works on clone/snapshot, no exception
ConcurrentHashMap	CAS for empty buckets, synchronized per-node for occupied
G1 GC	Region-based, collects most-garbage regions first, predictable pauses
Memory Leak	Heap dump → Eclipse MAT → check static collections, unclosed resources
@Component vs @Service vs @Repository	All beans, @Repository adds exception translation
@Transactional	AOP Proxy, self-invocation bypasses it, rollback on RuntimeException only by default
Circular Dependency	Redesign > @Lazy > Events